

Chronos Branching: High-Level Introduction

****Status:**** Draft

Summary

Chronos branching is a state management layer for agentic applications. Its vision is to provide one unified branching abstraction across many state stores: relational databases, filesystems, sandboxes, vector stores, object stores, and other application-managed state. Agents should be able to fork, mutate, evaluate, compare, discard, and merge speculative execution paths without reasoning separately about each store's native snapshot mechanism.

Chronos is designed to layer on top of TAR, the Transactional Agent Runtime. TAR provides the execution substrate: tool coordination, transaction boundaries, commit/abort, savepoints, and recovery across stateful tools. Chronos adds a branching abstraction above that substrate: instead of treating an agent run as one linear transaction, Chronos exposes a logical history tree of states.

The key design choice is to decouple branching from database transactions. Databases still provide the serial, durable, isolated execution primitive. Chronos uses those transactions to update branch metadata and branch-local data atomically, while presenting agents with Git-like branching semantics.

For relational data, Chronos is bolt-on: applications keep using SQL over logical tables, and Chronos rewrites queries to expose the state of the checked-out branch.

Why Agentic Systems Need Branching

Agentic applications are increasingly stateful. They update short-term memory, long-term memory, databases, files, vector indexes, generated artifacts, and external environments. A reliable agent runtime therefore needs more than a single commit/rollback boundary. It needs structured state exploration.

Several properties make agentic execution different from traditional transactional workloads:

- Agents often discover errors late. When a trajectory fails, the useful action

is not always "rollback the last step"; it may be "return to the state before step 17 and try a different plan."

- Tool calls and sandboxed programs are stateful. A trajectory may mutate files,

databases, caches, generated code, indexes, and environment state. Restoring the sandbox exactly matters, but full sandbox checkpointing can be expensive.

- Exploration is non-linear. Agents may run multiple branches in parallel,

compare their outcomes, keep the best branch, and discard the rest.

- The state is heterogeneous. Agent state can include internal state such as

memory and plans, and external state such as relational data, multimodal data, files, and environment state.

The BranchBench paper, "BranchBench: Aligning Database Branching with Agentic Demands" (arXiv:2604.17180), argues that agentic workloads create a branch-mutate-evaluate loop over database state. It identifies representative workloads such as agentic software engineering, failure reproduction, data curation, Monte Carlo Tree Search, and simulation. These workloads need fast branch creation and deletion, scalable deep and wide branch trees, efficient

reads and writes on branch state, cross-branch comparison, and isolation between speculative worlds.

Chronos adopts this view, but broadens the scope from database branching alone to agent state management across heterogeneous stores. The goal is a system that supports CRUD over branchable application state with consistency and reliability guarantees, whether that state lives in relational tables, files, sandboxed program state, vector indexes, or other state systems.

Why Transactions Alone Are Not Enough

A database transaction models a serial unit of work:

```
begin -> read/write -> commit
      |
      +--> rollback staged work
```

This is useful, but it does not directly model an exploration tree. Transactions and savepoints let an application time travel inside one linear execution order. Agent exploration often wants a durable tree:

```
main
|
+-- attempt_a
|   |
|   +-- attempt_a1
|   |
|   +-- attempt_b
|       |
|       +-- attempt_b1
```

Nested transactions can simulate part of this model. For example, best-of-N exploration can create subtransactions, commit the winner, and roll back the losers. But that is still a poor fit for general agentic branching:

- Branches are not first-class named states.
- Parallel branches are awkward to keep alive concurrently.
- Rollback is tied to a stack-like execution order.
- Long-running transactions are problematic because LLM calls and tool

execution may take seconds or minutes.

- MVCC garbage collection and locks can be harmed by long-lived speculative transactions.

- Heterogeneous state such as files, vector stores, and sandboxes does not share one native transaction manager.

Chronos therefore treats transactions as an implementation primitive, not as the user-facing branching abstraction.

Layering With TAR

TAR provides reliable execution over tools and state systems. Chronos branching layers on top of TAR to give agents a persistent logical state tree.

Agentic application

- planning
- evaluation
- branch selection
- policy checks

Chronos branching layer

- branch creation and checkout
- branch-local CRUD
- checkpoints
- diff, compare, merge
- branch garbage collection

Predicate rewrite and state adapters

- SQL query rewrite
- filesystem/sandbox overlays
- vector or memory overlays
- store-specific visibility rules

TAR transaction layer

- begin / commit / abort
- savepoints
- tool enrollment
- recovery and coordination

Underlying state systems

- PostgreSQL / SQLite
- filesystems and sandboxes
- vector stores
- external services

The branching layer gives agents one logical model across many physical storage systems. The TAR layer ensures each mutation to that model is applied safely.

System Model

Chronos separates two concerns:

Database/TAR transaction layer

- serial execution
- atomic commit and rollback
- isolation and durability
- locks, indexes, constraints, recovery

Chronos branching layer

- named branches
- arbitrary branch tree topology
- branch-local reads and writes
- checkpoints
- branch comparison and merge

A normal database transaction opened through a checked-out Chronos branch mutates only that branch. If the database transaction commits, the branch state advances. If it rolls back, the branch state is unchanged.

This lets Chronos preserve ordinary database correctness while exposing a richer state model to the agent.

Relational Branching In One Picture

The relational design has three conceptual layers:

```
User SQL
  SELECT * FROM products WHERE sku = 'abc'

Predicate rewrite layer
  SELECT *
  FROM physical_products
  WHERE sku = 'abc'
    AND live_lo <= :branch_point
    AND :branch_point < live_hi
    AND deleted = false

Interval branching model
- branches own segments in integer space
- row versions get intervals from the segment that wrote them
- branch points are read positions inside branch segments
- updates split old row intervals to preserve isolation
```

Applications continue to use logical table names. The branch session determines which branch state those table names refer to.

Interval Branching Model

The interval model represents branching with a number-line model:

- Every branch owns a monotonically shrinking segment $[lo, hi)$ in an integer space.
- Each branch also has a representative read point inside that segment.
- Every physical row version owns a visibility interval $[live_lo, live_hi)$.
- A row version gets its interval from the segment of the branch that wrote it.
- One physical row version can be shared by many branches when its interval covers their read points.

Reads use the branch's representative point:

```
row.live_lo <= b.branch_point < row.live_hi
AND row.deleted = false
```

Chronos adds this predicate to every user query over a registered logical table. Reads do not walk the branch lineage. They evaluate one branch point against row intervals.

Writes use the branch's current segment:

```
new_row.live_lo = branch.segment.lo
new_row.live_hi = branch.segment.hi
```

This is the missing link between branches and rows: a row written by branch X is visible over X's current segment. If a child is later forked from X, and that row interval covers the child segment, the child inherits the same physical row without copying it.

Branching

Each branch owns a monotonically shrinking segment of the integer space. When Chronos forks a child branch, it splits the parent branch's current segment:

Before:

```
parent segment
[-----)
```

After fork:

```
parent continuation
[-----)

child segment
[-----)
```

The parent continues in one subsegment. The child receives the other subsegment. The parent's future segment is smaller than its previous segment, so a branch's owned segment shrinks monotonically as it forks children. Each segment receives an interior read point. This partitioning recurses as branches fork from branches.

Branch creation is metadata-only. No user rows are copied when a branch is created. Existing row intervals are left unchanged, so rows written before the fork can remain shared by both sides of the fork.

Writes

Writes are the step that assigns row intervals and maintains logical branch isolation.

When a branch inserts a new logical row, Chronos creates a physical row version with the branch's current segment as its visibility interval:

```
branch segment:
[20, 40)

insert value B:
  B over [20, 40)
```

When a branch updates an inherited row, Chronos cannot simply add the new row. It must also remove the writer's segment from the old row's interval; otherwise the branch point could see both the old and new versions.

For a given logical primary key and branch, Chronos maintains this invariant:

```
At most one live physical row is visible for that logical key in that branch.
```

When a branch updates or deletes a logical row, Chronos finds row versions whose intervals overlap the branch segment and splits them as needed.

Example:

```
old row:
  value = A
  visible over [0, 100)
```

branch writes value B over [20, 40)

after write:

A over [0, 20)
B over [20, 40)
A over [40, 100)

For delete, the middle interval becomes a tombstone:

A over [0, 20)
deleted tombstone over [20, 40)
A over [40, 100)

This is why Chronos can avoid copying full tables at branch creation time while still giving each branch an isolated logical table view.

The overall process is:

1. Branch creation splits only branch metadata.
2. New rows receive the writer branch's current segment as their interval.
3. Reads test one branch point against row intervals.
4. Updates/deletes split overlapping old row intervals and install a replacement over the writer's current segment.

Guarantees and Design Goals

Chronos aims to provide:

- Branch isolation: writes to one branch are not visible in sibling branches.
- Cheap forks: branch creation updates metadata rather than copying user rows.
- Transactional branch mutation: each branch-local SQL transaction commits or rolls back atomically.
- Unified branching: agents use one branch/checkpoint/diff/merge abstraction across relational data, files, sandboxes, vector stores, and other state state systems.
- Constant-shape read visibility: reads use a fixed predicate independent of branch depth.
- Shared physical storage: unchanged rows are shared across branches.
- Explicit state management: branches, checkpoints, diffs, and merges are application-visible concepts.

These guarantees assume that all relevant application tables are registered with Chronos and accessed through branch-bound sessions.

Trade-Offs

Chronos intentionally chooses a bolt-on design. That makes it deployable over existing databases and compatible with ordinary SQL, but it creates trade-offs:

- Query performance depends on the SQL optimizer and indexes over visibility metadata.

- Write-heavy branching can fragment physical rows because updates split intervals.

- Fixed-width interval spaces require careful allocation, relabeling, or depth limits.

- Branch-local schema changes are outside the first relational design; the initial model branches row contents under a shared schema.

- Cross-store consistency depends on TAR coordination and state adapters.

These are different trade-offs from systems that implement branching in the storage layer, WAL layer, or a custom content-addressed storage engine. Chronos prioritizes a portable logical branching layer for agent state management.